

Automated Test Case Generation from Unstructured Software Requirements Using Advanced NLP Techniques

Nidhi Bhatia

Department of Computer Science and
Engineering,

SRM Institute of Science & Technology,
Delhi NCR Campus, Ghaziabad, (U.P.), India
nc1222@srmist.edu.in

Jitendra Singh

Department of Computer Science and
Engineering,

SRM Institute of Science & Technology,
Delhi NCR Campus, Ghaziabad, (U.P.), India
jitendr@srmist.edu.in

Vandana

SCRIT, CCS University,
Meerut (U.P.), India
ranavd@gmail.com

Abstract- Natural Language Processing (NLP) has emerged as a transformative tool in software engineering, particularly in automating test case generation (ATCG). Traditional ATCG approaches are limited to structured requirements (e.g., Cause-Condition-Effect formats), leaving unstructured, real-world specifications unaddressed. This paper proposes a novel NLP-based methodology to extract test cases from unrestricted software requirement documents (SRS) using a hybrid pipeline of language models (BERT, GPT) and rule-based techniques (NER, dependency parsing). Our approach addresses ambiguity and domain variability by integrating semantic analysis, constraint extraction, and machine learning-based validation. Experimental results across three domains (University, HR, and Coal Mine Management Systems) demonstrate 85–87% accuracy and 30–40% effort reduction compared to manual generation, while identifying key challenges like implicit requirements and contextual gaps. This work bridges the critical gap between unstructured client inputs and actionable test cases, enabling early-stage testing in Agile workflows.

Keywords- ATCG-Automatic Test Case generation, NER-Named entity recognition, NLP-Natural Language Processing, System Under Test, Unstructured Requirements, Test Automation, Semantic Parsing

I. INTRODUCTION

NLP can be termed as “Making human languages understandable, generatable, readable, processable and translatable by machines.” This technology is being used in almost every area where text, speech, sentiments, and processing of structured/unstructured data is involved. Some of its application areas are generation of predictive words, email filters, data analysis, sentiment analysis, smart assistants, text summarization and the list goes endless. Python language plays the key role in execution of these tasks. Software testing domain relies heavily on test cases being generated from the software requirement specification document. There are several tasks in testing domain which involves the need of natural language processing like generating test cases, generating test data, preparing test plan, identifying duplicacy in test cases, recognition of named entities etc. Software quality is the main concern of any organization. By generating the automatic test cases at such an early stage using SRS document, will definitely initiate the testing process at an earliest (Before actual coding or designing of product gets started) leading us to know the drawbacks or lacunas in requirement document, it will also reduce the time taken by test engineers in manually writing the test cases. Previous studies done by Z. Khaliq et al., recognize the activities

involved in software testing where AI has made a substantial influence and critically improved the methods for every activity involved in manual testing making it automated and fast to perform [1]. ATCG will require almost all of the five phases of NLP. Which include lexical analysis followed by syntax analysis then semantic analysis, there after discourse and pragmatic analysis. Further the SRS document has to be synthesized to generate various types of test cases ranging from functional, UI cases, security cases mentioned in the client’s requirements document. Any requirement which is missing will not be generated as a test case, this research paper only deals with requirements which are stated in text. ATCG will save tester’s efforts up to almost 40-50% of overall STLC duration. It will also automate the most time-consuming phases of STLC. It is impractical to automatically generate test cases with 100% efficiency because test case generation is a complex task. The requirement document may have missing conditions. It is worth mentioning that defining or preparing data for testing techniques like partition equivalence classes, checking boundary values, penetrating into SUT, and ensuring its security is a challenging task for an automated system. NLP can assist in this task but can never replace the domain expertise of quality engineers in writing test cases which are beyond the scope mentioned in SRS.

II. CONTRIBUTION OF THE STUDY

- A. Propose a novel methodology for ATCG task
- B. Applying ATCG on a sample set of requirements from various domains and authenticate the efforts saved by ATCG in comparison to manual generation of test cases
- C. Identifying the issues and challenges in ATCG

III. RELATED WORK

Till now in software industry test cases are written much later when the prototype/functionality of the first version of software is released. J. Fischbach et al. suggest writing requirements in a rule based *if-then* structure. Where *it* becomes the functionality of the test and *then* becomes the expected output. Client’s requirements of functionality often define system behaviour by linking events to each other [2]. J Kocerka et al. mentioned that this format can easily be converted to test cases saving a lot of tester’s effort. But requirements written in such manner are considered as restricted or given in casual form. SRS document is generally domain specific hence it is easier to develop an automatic process to identify the keyword phrases of requirements

written in human understandable languages [3]. In their study Srinivas Perala, et al. used automated test tools like NI Test Stand, dSPACE and Automation Desk, ZIRA are being used in industry for other phases of STLC like ATSG (Automated Test Scripts Generation), generating test data, test management, test cases recording, test reporting etc. For several appliances, many test automation techniques have been tried which employ embedded software. They have also presented a unique method to choose the necessary test cases from enormous test suites so that time and cost of testing can be reduced. [4]. V. Preethi et al. said that these tools and suggested frameworks use editor to manually write test cases and then further automatically generate test suits using AI & ML. If it is required to explore documents in a broader manner computer must understand human readable languages. For this purpose, we can make use of both machine learning and natural language processing algorithms [5]. Masuda, S. et al, used Syntax-tree resemblance for deriving test-cases from software requisites [6], but not all requirements are of the form condition-action. In order to generate test cases, I. Raharjana I. et al. suggested to employ user stories as semi-structured arrangement of instructions or requirements. It consists of three aspects, namely features of who (the actor), what (action performed), and why [7]. R. Gropler et al. used Semi-automated techniques for requirements-based model generation and proposed that they decrease human work and support recurrent changes in requirements and addition in requirements [8]. V. Garousi et al. have confirmed many methods to reduce the human efforts of extracting test cases from user requirements [9]. P. Singh's patent uses different ML models like neural networks, recurrent neural networks, random forest, scorecards, support vector machines, logistic regression, convolutional neural networks and many more to automate the test cases [10]. When an input SRS is given to an NLP system the verbs may be the functionality applied on the actor which are nouns in the given SRS. A. Salman in his research deals only with first three phases of NLP i.e., morphological analysis, syntax analysis and semantic mappings for the generation of test cases. Discourse and Pragmatic analysis are not performed since they deal with domain restricted language specifications [11]. B. Chelliah et al. has used a novel technique of generating test cases involving two steps: define followed by design. Quality engineers make estimations, and diagnose or extract constraints in the define phase. The activities which are included in design phase are state the test, identify test data, unify all constituents in a sequence of tests which is known as a test suit. [12]. M. Pribisalic et al. in his study show that when test cases are extracted from activity diagram user can authenticate this process by counting the number of test cases extracted manually, evaluating time taken in comparison to the ones which are generated using automated method [13]. C. Wang et al. creates acceptance test cases using NLP from use case models; we can infer the phrases from use case specification thereafter are secondary task is to generate OCL (Object Constrained Language) [14]. During project testing phase user stories can be administered using NLP which later help in test case generation [15]. N. Mulla et al. in his patent used conditional random fields (CRF) for mining information of actor's, actions, and conditions. K. Kikuma et al. have suggested that information extraction from design documents involves increase in accurateness of extraction by preparing a huge amount of training data to build a machine learning model [16]. S. Allala et al. illustrate various approaches of model-to-model conversions of user requisites to test cases

have been shown previously [17]. G. Mahalakshmi et al. reveals that NER plays an important role in ATCG task. Identify various entities and then possible states that entity can acquire. Example an entity "Username" can have values which is correct or incorrect or may be blank [19]. A. Azzazi et al. in his study of user stories to use cases show that any NLP module extracts nouns as actors and the extraction of verbs is done for defining functions or actions [20]. A. Ansari et al. illustrates that functional requirement written in any form can aid in creation of the test cases [21]. P. Andrejko et al. suggested that we can also perform grammar tree transformations on selected area of corpus to generate the test cases [22]. S. Masuda et al. shows software testing's efficiency and effectiveness depends upon creating test cases. The primary task of a test engineer is to create test cases from specification documents. Engineer's expertise or domain knowledge is a pre requirement for creating test cases [23]. M. Robeer et al. demonstrates that to create conceptual models from user stories we can incorporate NLP heuristic into an algorithm [24]. S. Masuda in his study makes use of design documents for generating test cases. These documents are given as input to document analysis tool [25]. D. Leitaio et al. presented an innovative NLP tool to translate test cases descriptions into formal specifications [26]. T. Dias et al. in their study show that automated software testing has proved its efficiency in saving time, resources, and expenses by automating the most mundane tasks, enhancing software testing, and uplifting the overall quality of software [27]. S. Zirape et al. also claims to create a corpus or a dictionary that may be used for generating the test cases which was previously produced manually but now using the NLP techniques they aimed to produce the dictionary automatically [28]. R. Reyna et al. in their study prove that a software tester needs test case as tools to know which steps to trail, and so an unambiguous and clear definition is critical to ensure that software meets customer needs, meets security needs and accomplishes its probable purposes [29]. S. Konreddy shows in his study that a collection of test cases is automatically derived from the documented specifications after a sequence of Natural Language Processing steps. Although, the accuracy of such a process is not flawless, imposing the use of a professional tester for peer inspections [30].

IV. METHODOLOGY

In our study we take into consideration software requirements that are written in English language and can be of any domain, having an unrestricted (free) form. In process of ATCG using NLP following steps need to be performed

A. Take any SRS document as input.

The SRS document should contain clear, concise requirement statements including module specification, functional requirements, constraints on functionality, non-functional requirements, external interface requirements and other requirements lying in the category of security, maintainability, or usability etc. Extract all textual data (Any missing requirement lies out of scope for this study)

B. Perform Data preprocessing steps like tokenization, lemmatization/stemming, removal of stop words and other data cleaning techniques

Segment the requirement text into sentences by applying tokenization. Tokenization means to break paragraphs into sentences, sentences into words and phrases for easier processing. We can use NLP libraries like NLTK, SpaCy,

TextBlob for tokenization, removal of stop words and punctuations. Further if in any case it is required to substitute words with its base or root form, we can apply lemmatization/stemming.

C. POS tagging using python libraries available.

Label each word with an appropriate part of speech tag (to identify its form a noun, verb or an adjective etc in the sentence. We can use Flair, SpaCy for POS tagging)

D. Identify the keywords in each sentence using various keyword extraction algorithms like RAKE, YAKE and TF-IDF etc. [18]

This step helps us to extract actors, actions, objects, data fields in the SUT.

E. Apply Information extraction techniques on document to identify functionality, inputs constraints and output.

In this step we analyse the grammatical structure of the sentence to extract relationships of the kind subject-action-object. We can use Stanford parser for dependency parsing. Then apply semantic parsing to extract the meaning, intent, entities, and logical conditions for test scenarios. We may use pre trained large language model like BERT, GPT for semantic parsing and test cases generation giving requirements as prompts for ATCG in a stated format.

F. Generate a unique test case of requirement stated and also extract and add constraints given in SRS.

This step helps us to extract rules and constraint from the requirements. Extracted information is formatted into predefined test case template. and test cases are generated.

NLP has the capability to infer negative test cases which are not mentioned in the requirements using techniques like constraint analysis, logical inference from positive cases, dependency parsing pattern matching etc.

Using the methodology described above, we can integrate machine learning frameworks to build and train a model specifically for Automatic Test Case Generation (ATCG). By employing frameworks such as TensorFlow, PyTorch, and Scikit-learn, along with the NLP techniques discussed earlier—such as tokenization, POS tagging, keyword extraction, and semantic parsing—we can develop an end-to-end learning-based system capable of generating test cases from natural language requirements.

The model can be trained on annotated SRS datasets, where each requirement is mapped to corresponding test cases. Over time, the system learns patterns in how different types of requirements (e.g., functional, constraint-based) are transformed into structured test steps. This not only improves the efficiency of test case creation but also reduces dependency on manual efforts and domain expertise. Figure 1 shows the step wise NLP pipeline for ATCG task.



Fig. 1. NLP pipeline for ATCG task

TABLE I. SAMPLE REQUIREMENTS WRITTEN IN NATURAL LANGUAGE FOR ATCG

A user logs in to the BUY application using correct username and password. User Clicks on the choice menu and selects single or multiple items from the store. The items are added to the cart and the cost of each item multiplied by number of units is displayed in the bill. Then the User proceed to the payment gateway for paying the bill. The choices of payment can be a credit card, debit card or cash. After payment, the User gets the status of bill as Pending or Successful. Then the User can logout using logout link.

TABLE II. ILLUSTRATES THE TEST CASES MANUALLY WRITTEN FOR SAMPLE REQUIREMENTS GIVEN IN TABLE I

Test Case ID	Test Case Description	Test Data	Expected Output
TC1.	User [Correct] Username [Correct] Password.	Username and Password	User Logs in to BUY application
TC2.	User Clicks on Choice menu.	NIL	Choice menu opens
TC3.	User Selects single or multiple items from store	NIL	Items get selected
TC4.	The items are added to cart	NIL	Items get added to cart
TC5.	Items multiplied by the number of units displayed in bill	NIL	Bill is generated
TC6.	User proceeds to payment gateway for paying the bill	NIL	Payment gateway is launched
TC7.	The choice of payment can be a credit card, debit card or cash	NIL	Payment choices are seen
TC8.	After payment, the user gets the status of bill	NIL	Bill status Pending or Successful
TC9.	User logs out using logout link	NIL	Logs out

V. RESULTS

We used SRS of students projects in four different domains which comprised of University Management System, Human Resource Management System and Coal Mine Management System. We applied the proposed methodology of generating test cases combining NLP, rule based and pre trained large language models to generate test cases automatically. Table 3 displays comparison of ATCG against manual test case generation using Requirement Traceability Matrix (RTM)

TABLE III. COMPARES THE PERFORMANCE OF AUTOMATIC TEST CASE GENERATION (ATCG))

Scenario	Method	Requirements Coverage (%)	Execution Time (min)	Effort Saved (Human-Hours)	Accuracy (%)
University Management System	ATCG (NLP-based)	65	30	25	85
	Manual Generation	90	180	0	95
Human Resource Management	ATCG (NLP-based)	62	40	30	82
	Manual Generation	88	200	0	94
Coal Mine Management System	ATCG (NLP-based)	68	35	28	87
	Manual Generation	92	240	0	96

VI. OBSERVATIONS

1. **Requirement Coverage:** ATCG covers more than 60% of requirements consistently, which is a significant contribution considering the efforts saved.

2. **Time Efficiency:** ATCG is much faster, reducing execution time by over 80% compared to manual generation.
3. **Effort Savings:** Automation saves substantial human effort in writing test cases.
4. **Accuracy Trade-off:** Manual test cases have slightly higher accuracy but at a much greater time cost.

TABLE IV. COMPARES OUR METHODOLOGY WITH RULE BASED AND PROMPT BASED ATCG

Method	Precision	Recall	F1-score	Human Effort Saved
Our Methodology (NLP+ML)	0.88	0.86	0.87	High
Rule-Based	0.70	0.65	0.67	Medium
GPT-3 Direct Prompt	0.83	0.77	0.80	Medium

VII. ISSUES AND CHALLENGES

1. **Ambiguous or Incomplete Requirements:** If the requirement document contains ambiguities or lacks necessary details, it can lead to test cases that are vague or insufficient to cover the desired functionality. Automated generation may not be able to resolve these ambiguities on their own and require human expertise and intervention.
2. **Lack of Contextual Understanding:** Test case generation algorithms may not possess the contextual understanding required to interpret requirements accurately. As a result, they may generate test cases that do not align with the intended behavior or fail to consider important edge cases.
3. **Implicit Requirements:** Some requirements may be implicit and not explicitly stated in the document. These implicit requirements may be missed during automated test case generation, leading to inadequate test coverage.
4. **Unrealistic or Infeasible Test Cases:** Automated generation algorithms may produce test cases that are not practical to execute or require complex set-up and environment configurations. Such test cases may be difficult to execute or maintain, making them less valuable in practice.
5. **Complex Scenarios and Dependencies:** Certain requirements may involve complex scenarios or dependencies between different functionalities. Automatically generating test cases that cover such scenarios accurately can be challenging, as the algorithm needs to be understood and represent these dependencies correctly.
6. **Non-Functional Requirements:** Test case generation algorithms often focus on functional requirements, such as input-output scenarios. Non-functional requirements, such as performance, security, or usability, may not be adequately addressed through automated test case generation alone.
7. **Validation of Generated Test Cases:** Automatically generated test cases need to be validated to ensure their correctness and effectiveness. This process may require manual intervention and human expertise to review, modify, or enhance the generated test cases.
8. **Limited Domain Knowledge:** The test case generation algorithms may have limited domain knowledge or lack domain-specific rules and constraints. This limitation can affect the correctness and significance of the test cases, especially in complex or specialized domains.
9. **Test Case Maintenance:** As the requirement document evolves or changes over time, the automatically generated test cases may become outdated or no longer align with the updated requirements. Regular maintenance and synchronization between the requirement document and generated test cases are necessary to ensure their ongoing usefulness.

VIII. CONCLUSION

In this study, we proposed a systematic and intelligent methodology for automatic test case generation (ATCG) from software requirement specifications using Natural Language Processing (NLP) and Machine Learning (ML) techniques. Our approach is designed to handle SRS documents written in unrestricted natural language and works across diverse application domains.

By breaking down the process into sequential steps—starting from requirement preprocessing, POS tagging, and keyword extraction, all the way to semantic parsing and test case generation—we created a pipeline that not only automates test creation but also brings substantial time and effort savings. Furthermore, the integration of semantic parsing and machine learning allows the system to infer both positive and negative test cases, improving the depth and reliability of test coverage. Using practical scenarios such as University Management Systems, HR systems and Coal Mine management, we observed that while manual generation still outperforms in accuracy and coverage, the ATCG method significantly reduces execution time and manual effort—making it a valuable augmentation tool for testers. In particular, the effort saved (ranging from 25 to 30 human-hours per case) demonstrates the practical viability and efficiency of the proposed method.

This methodology lays the groundwork for a more scalable and intelligent software testing process. While there is still room for refinement—particularly in increasing requirement coverage and test case accuracy—our results clearly demonstrate the potential of NLP-driven automation in software quality assurance. We conclude from the study that though it is challenging to extract test cases using NLP techniques and does not give 100% efficiency but its time and effort saving for testers. This study will also support to conclude that there is no restriction on developing domain specific models of ATCG. This approach is generic and applies to almost any area of SUT.

IX. FUTURE SCOPE

For future studies we can use MBTCG (Model Based Test Case Generation). MBTCG involves creating a formal machine learning model based on the natural language requirements. Test cases may be then generated automatically using model. For example, the MBTCG tool can be trained for generating test cases that cover all input conditions given in requirement document and then can be tested for new requirements. In future NLP based ATCG can also overcome its limitations using advanced ML algorithms.

REFERENCES

- [1] Khaliq, Z., Farooq, S. U., & Khan, D. A. (2022). "Artificial intelligence in software testing: Impact, problems, challenges, and prospect." arXiv preprint arXiv:2201.05371.
- [2] Fischbach, J., Frattini, J., Vogelsang, A., Mendez, D., Unterkalmsteiner, M., Wehrle, A., & Wiecher, C. (2023). "Automatic creation of acceptance tests by extracting conditionals from requirements: NLP approach and case study." *Journal of Systems and Software*, 197, 111549.
- [3] Kocerka, J., Krześlak, M., & Gałuszka, A. (2022). "Ontology Extraction from Software Requirements Using Named-Entity Recognition." *Advances in Science and Technology. Research Journal*, 16(3).
- [4] Srinivas Perala, D. A. R. (2021). "A novel framework design for test case authoring and auto test scripts generation." *Turkish Journal of Computer and Mathematics Education (TURCOMAT)*, 12(6), 1479-1487.
- [5] Preethi, V. (2021). "Survey on text transformation using bi-lstm in natural language processing with text data." *Turkish Journal of Computer and Mathematics Education (TURCOMAT)*, 12(9), 2577-2585.
- [6] Masuda, S., Matsudani, T., & Tsuda, K. (2021, April). "Syntax-Tree Similarity for Test-Case Derivability in Software Requirements." In 2021 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW) (pp. 162-172). IEEE.
- [7] Raharjana, I. K., Siahaan, D., & Fatichah, C. (2021).
- [8] "User stories and natural language processing: A systematic literature review." IEEE access, 9, 53811-53826.
- [9] Gröpler, R., Sudhi, V., García, E. J. C., & Bergmann, A. (2021). "NLP-Based Requirements Formalization for Automatic Test Case Generation." In CS&P (Vol. 21, pp. 18-30).
- [10] Garousi, V., Bauer, S., & Felderer, M. (2020). "NLP-assisted software testing: A systematic mapping of the literature." *Information and Software Technology*, 126, 106321.
- [11] Singh, P. K., Jude, A., Gogia, I., & Ahuja, R. (2020). U.S. Patent Application No. 16/034,117.
- [12] Salman, A. (2020). "Test Case Generation from Specifications Using Natural Language Processing."
- [13] Chelliah, B. J., Shreya, K., Raha, A., & Sravya, M. (2020). "An optimized and unique methodology for software test case automation strategy." *International Journal of Advanced Trends in Computer Science and Engineering (IJATCSE)*, 9(2).
- [14] Pribisalic, M. (2020). "Automatic generation of test cases from use-case specification using natural language processing." 33rd Bled eConference-Enabling Technology for a Sustainable Society, 725-734.
- [15] Wang, C., Pastore, F., Goknil, A., & Briand, L. C. (2020). "Automatic generation of acceptance test cases from use case specifications: an NLP-based approach." *IEEE Transactions on Software Engineering*, 48(2), 585-616.
- [16] Mulla, N., & Jayakumar, N. (2020). "The potent combo of software testing and NLP." In ICDSMLA 2019: Proceedings of the 1st International Conference on Data Science, Machine Learning and Applications (pp. 1623-1632). Springer Singapore.
- [17] Kikuma, K., Yamada, T., Sato, K., & Ueda, K. (2019, December). "Preparation method in automated test case generation using machine learning." In Proceedings of the 10th International Symposium on Information and Communication Technology (pp. 393-398).
- [18] Allala, S. C., Sotomayor, J. P., Santiago, D., King, T. M., & Clarke, P. J. (2019, July). "Towards transforming user requirements to test cases using MDE and NLP."
- [19] In 2019 IEEE 43rd Annual Computer Software and Applications Conference (COMPSAC) (Vol. 2, pp. 350-355). IEEE.
- [20] N. Bhatia, J. Singh, and Vandana. "Applying Keyword Extraction Techniques on Customer's Feedback of a Software Product to Accelerate Agile Based Development," in *2024 7th International Conference on Contemporary Computing and Informatics (IC3I)*, Sep. 2024. doi: 10.1109/IC3I61595.2024.10829078.
- [21] Mahalakshmi, G., Vijayan, V., & Antony, B. (2018). "Named entity recognition for automated test case generation." *Int. Arab J. Inf. Technol.*, 15(1), 112-120.
- [22] Azzazi, Ahmad. (2017). "A Framework using NLP to automatically convert User-Stories into Use Cases in Software Projects." *International Journal of Computer Science and Network Security. VOL.17. 71-76.*
- [23] Ansari, A., Shagufta, M. B., Fatima, A. S., & Tehreem, S. (2017, February). "Constructing test cases using natural language processing." In 2017 Third International Conference on Advances in Electrical, Electronics, Information, Communication and Bioinformatics (AEEICB) (pp. 95-99). IEEE.
- [24] Andrejko, P. D., Freed, A. R., Murch, C. M., Nielsen, R. L., Nordland, J. M., & Rivero, H. R. (2017). U.S. Patent No. 9,606,980. Washington, DC: U.S. Patent and Trademark Office.
- [25] Satoshi Masuda. (2017). "Studies on Generation of Software Test Cases by Using Natural Language Processing."
- [26] Robeer, M., Lucassen, G., Van Der Werf, J. M. E., Dalpiaz, F., & Brinkkemper, S. (2016, September). "Automated extraction of conceptual models from user stories via NLP." In 2016 IEEE 24th international requirements engineering conference (RE) (pp. 196-205). IEEE.
- [27] Masuda, S., Matsudani, T., & Tsuda, K. (2016). "Automatic generation of test cases using document analysis techniques." *International Journal of New Technology and Research*, 2(7), 263462.
- [28] Leitao, D., Torres, D., & de Almeida Barros, F. (2007, July). NLPForSpec: "Translating Natural Language Descriptions into Formal Test Case Specifications." In SEKE (pp. 129-134).
- [29] Dias, T., Batista, A., Maia, E., & Praça, I. (2023). TestLab: "An Intelligent Automated Software Testing Framework." arXiv preprint arXiv:2306.03602.
- [30] Zirape, S., Sharma, S., Ali, I. H., Kumbhar, M., Nalawade, R., & Jansari, m. (2023). "Add Self-Learning Ability to NLP for Automatic Test Case Generation."
- [31] Reyna, R., & Simske, S. J. (2022, June). "How to generate and import functional test cases into project management software systems using natural language processing." In Archiving Conference (Vol. 19, No. 1, pp. 40-44). IS&T 7003 Kilworth Lane, Springfield, VA 22151 USA: Society for Imaging Science and Technology.
- [32] Konreddy, S. D. R. "The Impact of NLP on Software Testing." *Journal of University of Shanghai for Science and Technology*, ISSN, 1007-6735.